INSURANCE CLAIM REQUEST FORM

A PROJECT REPORT

Submitted by

ADITHIYA V

(241501010)

in partial fulfilment of the End Semester Examination for the course

CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR in

BACHELOR OF TECHNOLOGY

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "INSURANCE CLAIM REQUEST FORM" is the bonafide work of [ADITHIYA V] ([241501010.]), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester III, under my supervision.

SIGNATURE

[Name of the HoD]
HEAD OF THE DEPARTMENT
Department of Artificial Intelligence and Machine Learning
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

[Name of the Course Faculty]
COURSE FACULTY
[Designation]
Department of Artificial Intelligence and Machine Learning
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on [DD Month YYYY].

INTERNAL EXAMINER ***EXTERNAL EXAMINER***

DECLARATION

I, [ADITHIYA V] ([241501010]), hereby declare that the project report titled "INSURANCE CLAIM REQUEST FORM", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports and dashboard — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai

Date: [19] October 2026

SIGNATURE OF THE STUDENT

[ADITHIYA V]

[241501010.]

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman [Name], Chairperson [Name] and Vice-Chairman [Name] for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, [Name], Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank [Name], Head of the Department, Department of Artificial Intelligence and Machine Learning, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, [Name], [Designation], for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for the Zoho Creator platform and its learning resources, and I thank my family and friends for their unwavering support.

ADITHIYA.V

ABSTRACT

Claims officers routinely record claim requests, customer policy details and supporting documents, and use these records to know which claims are still pending and how much has been claimed. When this information is kept on paper forms, in e-mails or in separate spreadsheets, it becomes scattered, the status of each claim has to be tracked by hand, and totals and status counts must be calculated repeatedly whenever a summary is needed. The Insurance Claim Request Form is a simple claim management application developed on the Zoho Creator low-code platform to bring these activities into one place. The application is organised into four modules: Dashboard, Requests, Customer Profiles and Claim Attachments. Structured forms capture claim requests (claim reference, claimant name, date of incident, claim amount, incident description, claim status, supporting document, submission date and policy number reference), customer profiles identified by policy numbers and claim attachments, and lookup fields link claims to customer profiles and attachments to claims. Each claim carries a status of Pending, Approved or Rejected. List reports display the stored records, and the dashboard summarises the total claim amount, the number of requests, customer profiles and attachments, the number of claims per status and a chart of claim amount by status. For the five sample claims, the dashboard shows a total claimed amount of $ 13,241.50 with four claims approved and one pending. The dashboard analysis is descriptive: the application does not use a trained artificial intelligence or machine learning model, and no external integrations are used.

Test cases covering data entry, record linking, dashboard totals, status counts and the chart have been defined, and the cases verified from the live application are reported in Chapter 6. The project demonstrates how the forms, lookups, reports and dashboard features of Zoho Creator reduce the amount of conventional code needed for a practical claim record- keeping application, while showing that correct data modelling, validation and testing remain essential.

Keywords: Low-Code Development; Zoho Creator; Insurance Claim Management; Claim Status Tracking; Lookup Fields; Dashboard

PROJECT AT A GLANCE

Attribute	*Details*
Project Title	INSURANCE CLAIM REQUEST FORM
Application Domain	Insurance – Claim Request Management
Student Name & Register No.	[ADITHIYA. V] – [241501010.]
Zoho Creator Edition	Zoho Creator Trial Edition
Application Name (Link Name)	Insurance Claim Request Form (link name: insurance-claim-request-form)
Published / Live URL	https://creatorapp.zoho.in/vadhi637/insurance-claim-request-form
Demo Credentials (per role)	Claims Officer / Application Owner: [vadithiya6@gmail.com]
Number of Forms / Subforms	3 forms (Claim Request, Customer Profile, Claim Attachment) / 0 subforms
Number of Reports (by type)	List: 3 (All Requests, All Customer Profiles, All Claim Attachments) Kanban: 0 Calendar: 0 Chart: 1 (Claim Status by Claim Amount on Dashboard) Others: 0
Number of Pages / Dashboards	1 (Dashboard)
Number of Workflows	Form: 2 (On Validate – Claim Amount Validation & Default Status / Submission Date) Schedule: 0 Approval: 0 Blueprint: 0 Batch: 0
Deluge Functions / Custom Actions	2 Deluge On Validate scripts (DS-01 Claim Amount Validation & DS-02 Default Status and Submission Date)
Integrations	None (not implemented in the current version)
Roles & Profiles Defined	Application Owner / Claims Officer (Default Administrator access; custom roles not required for single-officer scope)
Environments Used	Development Environment (Single-environment Zoho Creator workspace)
Date of Production Deployment	October 2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES viii
LIST OF FIGURES ix
LIST OF ABBREVIATIONS x
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 3
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY 4
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 5
2.3 Low-Code versus Traditional Development 5
2.4 Literature Survey 6
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 7
REQUIREMENT ANALYSIS 8
3.1 Stakeholders and User Roles 8
3.2 Functional Requirements 8
3.3 Non-Functional Requirements 9
3.4 Hardware and Software Requirements 9
3.5 Use Case Model 10
SYSTEM DESIGN 12
4.1 System Architecture 12
4.2 Data Model 13
4.3 Form Design 14
4.4 Process and Workflow Design 16
4.5 User Interface and Navigation Design 17
4.6 Access Control Design 17
IMPLEMENTATION 19
5.1 Application Setup 19

5.2 Forms and Data Capture 19
5.3 Reports 20
5.4 Pages and Dashboards 21
5.5 Deluge Scripting 22
5.6 Workflows and Automation 23
5.7 Integrations 24
5.8 User Management and Role-Based Access 24
5.9 AI and Analytics Features 25
5.10 Pro-Code Extension 25
5.11 Mobile and Portal Access 26
TESTING AND DEBUGGING 27
6.1 Testing Strategy 27
6.2 Test Cases 27
6.3 Debugging Deluge Scripts 28
6.4 Defect Log 29
6.5 User Acceptance Testing 29
DEPLOYMENT, SECURITY AND GOVERNANCE 30
7.1 Environments and Deployment Strategy 30
7.2 Performance Optimisation 31
7.3 Security Features 31
7.4 Governance and Administration 31
RESULTS AND DISCUSSION 33
8.1 Application Output 33
8.2 Achievement of Objectives 34
8.3 Comparison with the Existing System 35
8.4 Limitations of the Low-Code Approach 35
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 36
CONCLUSION AND FUTURE ENHANCEMENTS 37
9.1 Conclusion 37
9.2 Future Enhancements 37
9.3 Learning Outcomes and Career Relevance 38
REFERENCES 39
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 40
APPENDIX B – COURSE OUTCOME MAPPING 42
APPENDIX C – COMPLETE DELUGE SOURCE CODE 43
APPENDIX D – USER MANUAL 44

APPENDIX E – EVALUATION RUBRIC 45
APPENDIX F – VIVA-VOCE RECORD SHEET 47

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 5
Table 2.2: Summary of Literature Survey 6
Table 3.1: Stakeholders and User Roles 8
Table 3.2: Functional Requirements 8
Table 3.3: Non-Functional Requirements 9
Table 3.4: Hardware and Software Requirements 9
Table 3.5: Use Case Description – Submit Claim Request and Track Status..... 10
Table 4.1: Entities and Relationships 13
Table 4.2: Field Specification – Claim Request Form 14
Table 4.3: Field Specification – Customer Profile Form 15
Table 4.4: Field Specification – Claim Attachment Form 15
Table 4.5: Role–Permission Matrix 17
Table 5.1: Reports Implemented 21
Table 5.2: Deluge Scripts Implemented 22
Table 5.3: Workflows Implemented 23
Table 5.4: Integrations Implemented 24
Table 6.1: Test Cases 27
Table 6.2: Defect Log 29
Table 6.3: User Acceptance Feedback 29
Table 7.1: Deployment Checklist 30
Table 7.2: Performance Optimisation Results 31
Table 7.3: Security Controls Implemented 31
Table 8.1: Mapping of Objectives to Outcomes 34
Table 8.2: Existing System versus Proposed Application 35
Table A.1: Feature Coverage Checklist 40
Table B.1: Mapping of Course Outcomes to Report Chapters 42
Table E.1: Evaluation Rubric for the Individual Project 45
Table E.2: Performance Level Descriptors 45
Table F.1: Viva-Voce Questions and Responses 47
Table F.2: Marks Summary 47

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing (Manual) System 4
Figure 3.1: Use Case Diagram of the Insurance Claim Request Form 10
Figure 4.1: System Architecture of the Insurance Claim Request Form 12
Figure 4.2: Entity–Relationship Diagram of the Insurance Claim Request Form (Conceptual) 13
Figure 4.3: Workflow / Process Flow for Claim Request Entry and Status Tracking 16
Figure 4.4: Navigation Structure and Wireframe of the Dashboard Page 17
Figure 5.1: Application Home (Dashboard) and Navigation Sections 19
Figure 5.3: Claim Request Form in Live Mode 20
Figure 5.4: Requests Report with the Overview of Claim CLM-2026-004 21
Figure 5.5: Claim Attachments Report with Record Overview 21
Figure 5.8: Claim Status by Claim Amount Chart on the Dashboard 25
Figure 8.1: Scenario Step 1 – Opening the Claim Request Form 33
Figure 8.2: Scenario Step 2 – Claim CLM-2026-004 Recorded with Status Pending 34
Figure 8.3: Scenario Step 3 – Dashboard Totals, Claim Status and Chart after the Update 34

LIST OF ABBREVIATIONS

Abbreviation	*Expansion*
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
FK	Foreign Key
FR	Functional Requirement
IEEE	Institute of Electrical and Electronics Engineers
ML	Machine Learning
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho's AI assistant

CHAPTER 1

INTRODUCTION

1.1 Background

Insurance is built on claims. When a policyholder suffers a loss such as a car collision, water damage, a broken window or the theft of belongings, the policyholder submits a claim request to the insurer. A claims officer records the details of the incident, checks them against the policy, collects the supporting documents and finally decides whether the claim is approved or rejected.

In many small offices and academic settings these records are maintained on paper forms, e-mails and spreadsheets. Claim details, customer policy details and supporting documents therefore end up in different places, the current status of a claim has to be tracked by hand, and totals such as the number of pending claims or the total claimed amount must be calculated manually whenever a summary is needed.

The Insurance Claim Request Form has been developed as an individual academic project for the course CB23F35 – Low-Code Application Development using Zoho Creator. It is a small-scale application intended for a claims officer who wants to capture claim requests, customer profiles and claim attachments in one place and to see the status of all claims on a single dashboard. The main purpose of the project is to demonstrate how Zoho Creator forms, lookup fields, reports and dashboards can simplify claim request management.

The sample records used in this report — claimants named Sarah Miller, James Wilson, Emily Chen, Robert Taylor and Linda Garcia, and claim references CLM-2026-001 to CLM-2026-005 — are fictional demonstration data. They do not represent real customers, real policies or real claims.

1.2 Problem Statement

A claims officer needs a convenient way to record claim requests, link each claim to the customer's policy profile, keep the supporting attachments of a claim together and know at any time how many claims are approved, rejected or pending. When these activities are performed manually or across separate files, record keeping requires additional effort, claim status has to be tracked by hand, and a consolidated summary of claims is difficult

to prepare. A simple, centralised application is therefore required to manage claim request information and to present basic claim indicators in an understandable format. The specific pain points are:

- Claim, customer and attachment information is scattered across paper forms, e-mails and spreadsheets.
- The status of each claim (Pending, Approved or Rejected) is tracked manually and can easily become out of date.
- It is difficult to view the claim requests of a customer, or the attachments of a claim, together.
- Totals such as the total claimed amount and the number of claims per status are calculated by hand.
- There is no quick visual summary of the overall claim data.

1.3 Objectives

- To create a centralised form for recording claim requests (reference, claimant, date of incident, amount and description).
- To maintain customer profiles identified by policy numbers and to link them to claim requests.
- To store claim attachment details and link them to the corresponding claim requests.
- To record and track the status of each claim as Pending, Approved or Rejected.
- To provide reports and a dashboard showing total claim amount, total requests, total customer profiles, total attachments and claim status counts.
- To demonstrate the use of Zoho Creator's low-code development features in an insurance claim management application.

1.4 Scope of the Project

The Insurance Claim Request Form focuses on basic claim request management and claim status tracking for a single claims officer. The scope of the project includes:

- Claim request management (claim reference, claimant name, date of incident, claim amount, incident description, claim status, supporting document, submission date and policy number reference).

- Customer profile management, with each profile identified by a policy number.
- Claim attachment management (attachment name, file URL, upload date, file size and description) linked to claim requests.
- Reports for each module and a dashboard with summary cards, a claim status summary, a chart and quick links.

The application is accessed through a web browser on the Zoho Creator platform by the claims officer, who is also the application owner in the current version. Access through the Zoho Creator mobile app and a customer portal is not part of the current scope.

Out of scope. The application does not automatically decide whether a claim should be approved, does not detect fraud, does not calculate settlement amounts or make payments, does not use a trained machine learning model and does not integrate with any external system. Features that are not implemented are documented as future enhancements in Section 9.2.

1.5 Justification for a Low-Code Approach

A low-code platform lets developers build applications mainly by configuring visual models of data, user interfaces and logic, with hand-written code needed only for specific requirements [1]. Zoho Creator provides a visual form builder, automatically generated reports, a page builder for dashboards and a hosted cloud environment, with the Deluge scripting language available when custom logic is required [2].

The Insurance Claim Request Form is well suited to this approach. Its data consist of structured records (claim requests, customer profiles and claim attachments) related to one another through lookup fields, and its main requirements are data-entry forms, record lists, status tracking and a summary dashboard — all of which map directly onto standard Zoho Creator components. Building the same application in a conventional way would require a separately designed database, server-side code, user interface pages and hosting.

Using Zoho Creator therefore reduces the amount of conventional code to be written, removes the need to set up a server or database, and makes it easier to add or change fields later. A low-code platform does not remove the need for careful design: correct data modelling, validation of entered values, testing and access control still have to be planned and verified by the developer.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2

LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In a conventional approach, claim details, customer policy information and supporting documents are commonly maintained in separate paper records, spreadsheets or e-mail folders. The claims officer receives the claim details, notes them in a register or spreadsheet, looks up the policy details in another file, keeps the supporting documents in a separate folder, updates the status of the claim manually and then counts the claims by hand before preparing any summary. Figure 2.1 shows this general as-is process. It describes the general limitations of manual record keeping and is not a claim about the exact process followed at any particular organisation.

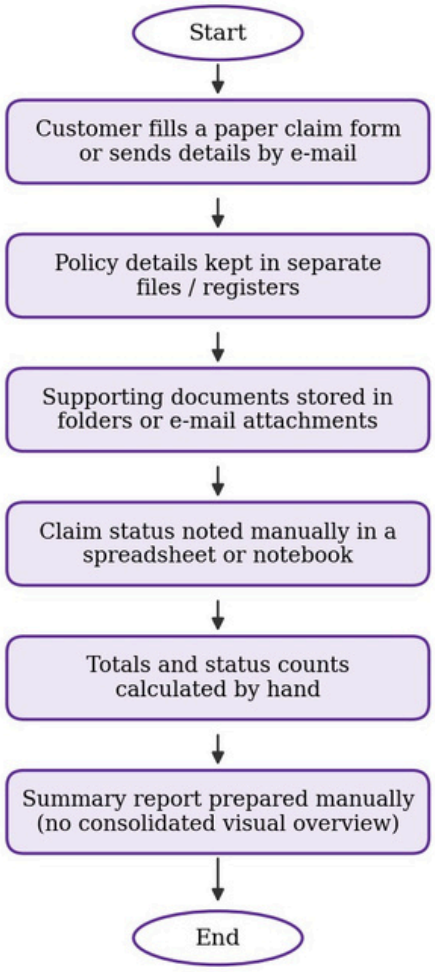


Figure 2.1: As-Is Process Flow of the Existing (Manual) System

2.2 Limitations of the Existing System

- ***Scattered information:*** claim, policy and document records are kept in different places.
- ***Manual status tracking:*** the status of every claim is updated and checked by hand.
- ***No combined view:*** it is difficult to see a claim, its customer profile and its attachments together.
- ***Repeated manual totals:*** claim amounts and status counts must be calculated by hand.
- ***Limited visual summaries:*** there is no ready overview of the claims handled.

2.3 Low-Code versus Traditional Development

Table 2.1 compares a conventional (pro-code) implementation of the Insurance Claim Request Form with its low-code implementation on Zoho Creator. The criteria are adapted from feature-based comparisons of low-code development platforms [3]. Development effort is described qualitatively because the time taken was not formally measured.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	*Traditional (Pro-Code) Development*	*Low-Code on Zoho Creator*
Development time	Longer: the database schema, server-side code, user interface pages and hosting must each be built and connected.	Shorter: forms, reports and the dashboard are configured visually; the actual effort was not formally measured.
Skills required	A programming language, a web framework, SQL and deployment skills.	Data modelling and Zoho Creator configuration; Deluge scripting only where custom logic is needed.
Hosting & infrastructure	A web server and database must be provisioned and maintained separately.	Hosted on the Zoho Creator cloud; no server set-up is required from the developer.
Mobile application	A responsive design or a separate mobile application must be developed.	Zoho Creator offers mobile access to applications; not tested for this project.
Security & access control	User authentication and authorisation must be coded and tested.	Uses Zoho account sign-in and platform permission settings (custom roles are not configured in this project).
Maintenance & change	Each change requires code modification, testing and redeployment.	Fields, forms and reports can be modified in the visual builder with little or no code.
Customisation limits	Fully flexible user interface and logic.	Limited to the components, layouts and scripting features offered by the platform.
Cost	Developer time plus separate hosting costs.	Depends on the Zoho Creator plan used; no separate hosting cost.

2.4Literature Survey

Thesources reviewedare studies of low-code development platforms [1], [3], [4] and the officialZoho Creatordocumentation [2], which inform how the Insurance Claim Request Formisbuilt. Table2.2summarises these sources.

Table 2.2: Summary of Literature Survey

S.No.	*Author(s) / Source & Year*	*Title / Product*	*Key Findings*	*Relevance to this Project*
1	A. C. Bock and U. Frank, 2021 [1]	Low-Code Platform	Describes low-code platforms as environments in which applications are built mainly by configuring models of data, user interfaces and processes, reducing hand-written code while tying the application to the concepts of the platform.	Explains why a form-and-report based application such as this claim request system suits a low-code platform, and the trade-offs involved.
2	A. Sahay et al., 2020 [3]	Supporting the Understanding and Comparison of Low-Code Development Platforms Low-Code	Analyses a selection of low-code development platforms and proposes a set of features for understanding and comparing them.	Provides the basis for the comparison criteria used in Table 2.1.
3	D. Di Ruscio et al., 2022 [4]	Development and Model- Driven Engineering: Two Sides of the Same Coin?	Discusses the relationship between low-code development and model-driven engineering, and the opportunities and challenges they share.	Helps explain that the application's forms act as models from which Zoho Creator generates the data store and user interface.
4	Zoho Corporation [2]	Zoho Creator Help Documentation	Official documentation of Zoho Creator forms, fields, reports, pages, workflows and user management.	Primary technical reference used to design and build the application.

2.5 Emerging Trends in Low-Code Development

AI-assisted application building. Low-code platforms increasingly offer AI-assisted features, such as generating application structures or scripts from natural-language descriptions. The forms, reports and dashboard of this application were configured by the developer; confirm and edit this sentence if any AI-assisted feature was used.

Citizen development. Low-code tools allow people with domain knowledge but limited programming experience to build simple applications. This project reflects the trend: its structure (claim requests, customer profiles, attachments) mirrors how a claims officer already thinks about claim files, so it can be understood and modified without deep programming knowledge.

Convergence with model-driven engineering. Low-code development is closely related to model-driven engineering, in which models are the main development artefacts [4]. In

this application the form definitions act as the model from which the platform generates storage and user interface.

Workflow automation. Organisations increasingly use workflow automation to reduce manual follow-up of business records. Status-based tracking, as used here for claims, is a simple first step towards approval workflows and notifications, which are listed as future enhancements.

2.6 Proposed System

The proposed system, the Insurance Claim Request Form, is a centralised Zoho Creator application that contains forms to capture claim requests, customer profiles and claim attachments, reports to display the saved records, and a dashboard to summarise the available information. Each claim request carries a Claim Status of Pending, Approved or Rejected, and is linked to one or more customer profiles through a lookup field. Claim attachments are linked to claim requests in the same way.

The dashboard summarises the application data with four cards (Total Amounts, Total Requests, Total Customer Profiles and Total Claim Attachments), a Claim Status panel showing the number of Approved, Rejected and Pending claims, a chart of Claim Status by Claim Amount, and quick links for adding new records. The dashboard is an indicative summary of the stored records; it does not decide whether a claim should be approved. The proposed system addresses the limitations listed in Section 2.2 as follows: scattered information is replaced by one application with related forms; manual status tracking is replaced by a Claim Status field that is summarised automatically; the reports provide a combined view of records; manual totals are replaced by dashboard summary cards; and the chart provides a visual summary.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	*Description*	*Key Responsibilities in the Application*
Claims Officer (Primary User)	The person who uses the application to manage claim requests.	Add and update claim requests, customer profiles and claim attachments; update claim status; view reports and the dashboard.
Application Developer / Administrator	The student developer who builds and owns the Zoho Creator application.	Design and maintain forms, fields, reports and the dashboard; manage application settings.
Customers / Claimants (Indirect Stakeholders)	Policyholders whose claims are recorded in the application.	No direct access in the current version; they benefit when the claims officer tracks the status of their claims.

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	*Requirement*	*Role(s)*	*Priority*
FR-01	The system shall allow the claims officer to add, view and update claim requests with Claim Reference, Claimant Name, Date of Incident, Claim Amount and Incident Description.	Claims Officer	High
FR-02	The system shall allow the claims officer to record the Claim Status of a request as Pending, Approved or Rejected.	Claims Officer	High
FR-03	The system shall record the Submission Date and Policy Number Ref of each claim request and allow a Supporting Document to be attached.	Claims Officer	Medium
FR-04	The system shall allow the claims officer to add, view and update customer profiles identified by a policy number.	Claims Officer	High
FR-05	The system shall allow a claim request to be linked to customer profiles through a lookup field.	Claims Officer	High
FR-06	The system shall allow claim attachment details (Attachment Name, File URL, Upload Date, File Size, Description) to be recorded and linked to claim requests.	Claims Officer	Medium
FR-07	The system shall reject a claim request whose Claim Amount is zero, negative or missing.	Claims Officer	High

FR ID	Requirement	Role(s)	Priority
FR-08	The system shall provide reports listing the records of each module.	Claims Officer	Medium
FR-09	The system shall display a dashboard with the total claim amount and the total number of claim requests, customer profiles and claim attachments.	Claims Officer	High
FR-10	The system shall display on the dashboard the number of Approved, Rejected, Pending and Total claims, a chart of Claim Status by Claim Amount and quick links to add or view records.	Claims Officer	Medium

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Forms and reports shall open within an acceptable time for a small data set (not formally measured in this project).
NFR-02	Security	Claim and customer data shall be accessible only to authorised Zoho account users of the application.
NFR-03	Usability	The interface shall use clear labels, a simple layout and a minimal number of fields per form.
NFR-04	Availability	The application shall be available wherever the Zoho Creator service and an internet connection are available.
NFR-05	Maintainability	Fields, forms and reports shall be modifiable through the Zoho Creator builder without rewriting the application.
NFR-06	Data Integrity	Claim amounts shall be positive values so that dashboard totals remain meaningful.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (Trial Edition)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment) – used on the Claim Request form On Validate event for amount validation and default status
Browser	A modern web browser such as Google Chrome or Microsoft Edge (latest version)
API testing tool	Not required (no API integration in the current version)
Other Zoho services	None used
Mobile	Not used in the current version
Client hardware	Desktop computer or laptop with keyboard, mouse and a reliable internet connection; no specialised hardware is required
Documentation	Microsoft Word or a compatible document editor for the project report

3.5 Use Case Model

Figure 3.1 shows the use case diagram of the application. The claims officer is the primary application developer/administrator who manages records and views the application. Customers are indirect stakeholders and do not interact with the application. The key use case is described in Table 3.5.

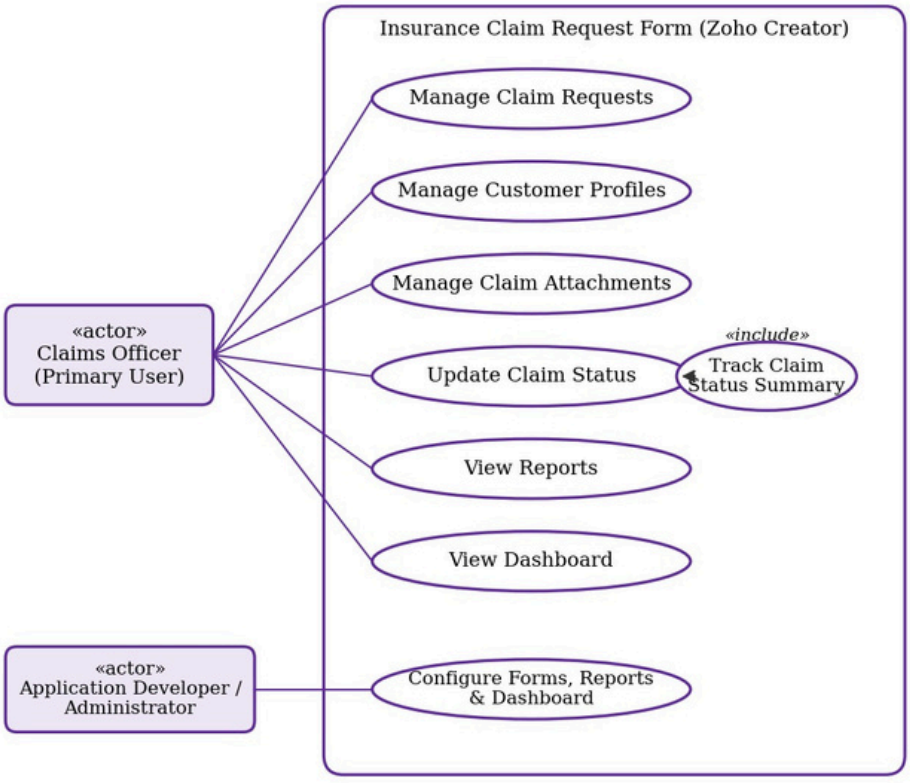


Figure 3.1: Use Case Diagram of the Insurance Claim Request Form

Table 3.5: Use Case Description – Submit Claim Request and Track Status

Use Case ID	UC-01
Use Case Name	Submit Claim Request and Track Status
Actor(s)	Claims Officer
Pre-condition	The customer profile for the policy exists in the application.

Main Flow	1. The claims officer opens the Requests section and the Claim Request form. 2. The officer enters the Claim Reference, Claimant Name, Date of Incident, Claim Amount and Incident Description. 3. The officer selects the Customer Profile and enters the Policy Number Ref. 4. The officer submits the form; the application validates the entries. 5. The claim is saved with its Claim Status and Submission Date. 6. The claim appears in the Requests report and the dashboard totals and status counts are updated. 7. After review, the officer updates the Claim Status to Approved or Rejected.
Alternate Flow	If the Claim Amount is zero, negative or missing, an error message is shown and the record is not saved.
Post-condition	A valid claim request is stored and reflected in the reports and the dashboard.
Related FR	FR-01, FR-02, FR-03, FR-05, FR-07, FR-09, FR-10

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

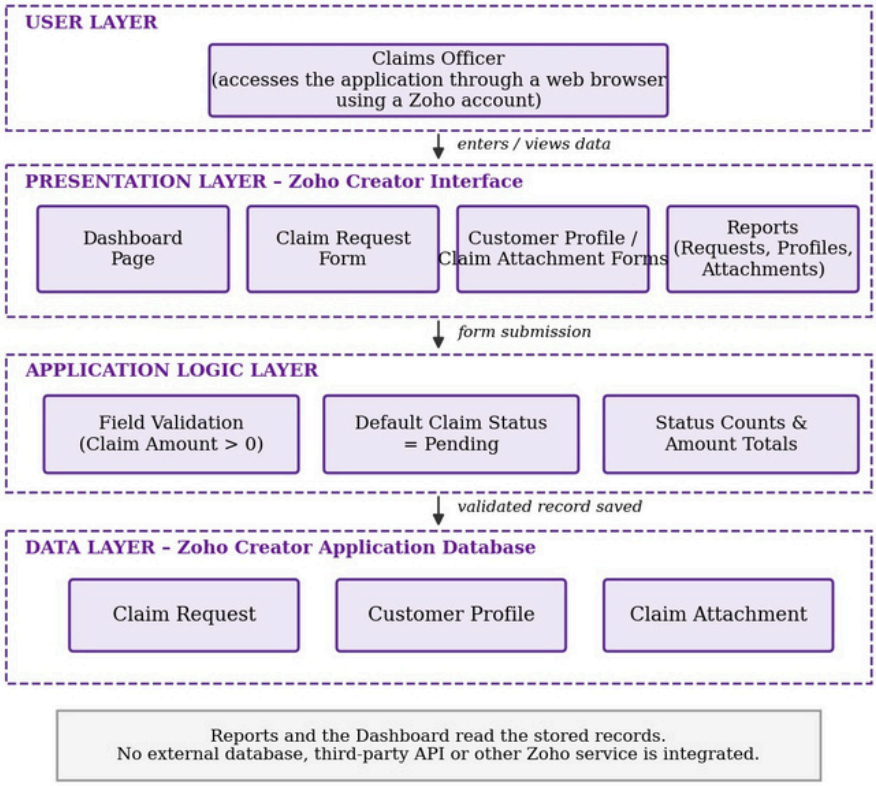


Figure 4.1: System Architecture of the Insurance Claim Request Form

Figure 4.1 shows the layered architecture of the application. The flow of information is: User → Zoho Creator Interface → Forms and Reports → Application Data → Dashboard.

User layer. The claims officer accesses the application through a web browser after signing in with a Zoho account.

Presentation layer. The Zoho Creator interface presents the Dashboard page, the data-entry forms and the reports. These are generated by the platform from the configured forms, reports and page.

Application logic layer. When a claim request is submitted, the entered values are validated and the claim is stored with its status. The dashboard summarises amounts and status counts from the stored records.

Data layer. Submitted records are stored in the built-in database of the Zoho Creator application, one record set per form (Claim Request, Customer Profile and Claim Attachment). The reports and the dashboard read these stored records. No external database or integration is used.

4.2 Data Model

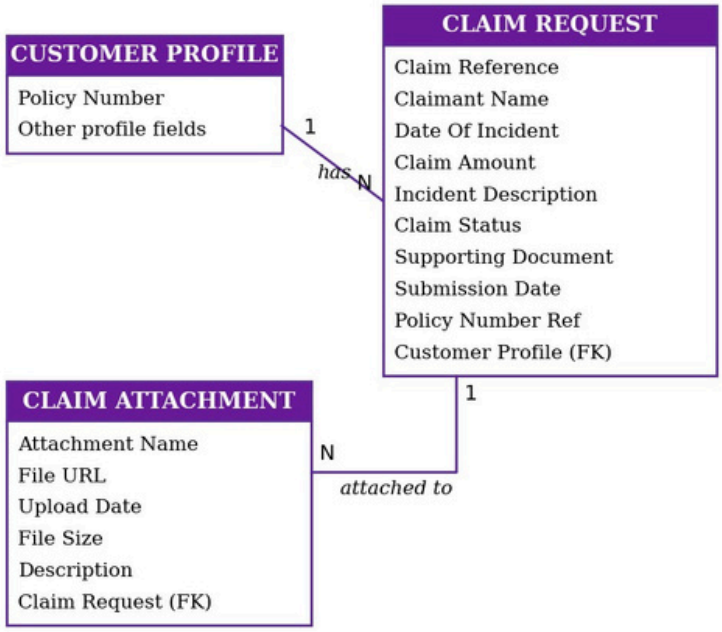


Figure 4.2: Entity–Relationship Diagram of the Insurance Claim Request Form (Conceptual)

The application has three entities, each implemented as a Zoho Creator form. A customer profile (identified by its policy number) can be associated with many claim requests, and a claim request can have many claim attachments. In Zoho Creator, such relationships are realised through lookup fields: the Claim Request form has a Customer Profile lookup and the Claim Attachment form has a Claim Request lookup. The relationships in Figure 4.2 and Table 4.1 represent the relational data model of the application.

Table 4.1: Entities and Relationships

Entity (Form)	*Key Fields*	*Related To*	*Relationship Type*	*Implemented Using*
Customer Profile	Policy Number (and other profile fields)	Claim Request	One-to-Many	Lookup to Customer Profile in the Claim Request form

Entity (Form)	*Key Fields*	*Related To*	*Relationship Type*	*Implemented Using*
Claim Request	Claim Reference, Claimant Name, Date Of Incident, Claim Amount, Claim Status, Policy Number Ref	Claim Attachment	One-to-Many	Lookup to Claim Request in the Claim Attachment form
Claim Attachment	Attachment Name, File URL, Upload Date, File Size, Description	Claim Request	Many-to-One	Lookup field (see above)

4.3 FormDesign

Tables 4.2to 4.4 give the field specification of the three forms. The link names shown are the standard field link names generated by Zoho Creator from the field labels. Check the Mandatory and Validation columns against your forms and correct them where they differ.

Table 4.2: Field Specification – Claim Request Form

Field Label	*Link Name*	*Field Type*	*Mandatory*	*Validation / Default*	*Remarks*
Claim Reference	Claim_Reference	Single Line	Yes	Unique reference (recommended)	e.g., CLM-2026-001
Claimant Name	Claimant_Name	Name	Yes	—	Name of the person making the claim
Date Of Incident	Date_Of_Incident	Date	Yes	Date picker, format dd-MMM-yyyy	Date on which the incident occurred
Claim Amount	Claim_Amount	Currency ($)	Yes	Must be greater than 0	Amount claimed
Incident Description	Incident_Description	Multi Line	No	—	e.g., Car collision, Water damage
Claim Status	Claim_Status	Drop Down	Yes	Choices: Pending / Approved / Rejected	Current status of the claim
Supporting Document	Supporting_Document	File Upload	No	—	Document supporting the claim
Submission Date	Submission_Date	Date-Time	No	Set to current date-time on submission	e.g., 20-Oct-2026 11:00:00
Policy Number Ref	Policy_Number_Ref	Single Line	No	—	e.g., POL-665544
Customer Profile	Customer_Profile	Lookup (Customer Profile)	No	Select existing profiles	Shows policy numbers such as POL-2026-001

Table 4.3: Field Specification – Customer Profile Form

Field Label	*Link Name*	*Field Type*	*Mandatory*	*Validation / Default*	*Remarks*
Policy Number	Policy_Number	Single Line	Yes	—	Displayed in the lookup, e.g., POL-2026-001
Other fields					Add the remaining fields of your Customer Profile form

Table 4.4: Field Specification – Claim Attachment Form

Field Label	*Link Name*	*Field Type*	*Mandatory*	*Validation / Default*	*Remarks*
Attachment Name	Attachment_Name	Single Line	No	—	Name of the attached document
File URL	File_URL	URL / Single Line	No	—	Location of the file
Upload Date	Upload_Date	Date	No	—	Date the file was uploaded
File Size	File_Size	Number	No	—	Size of the file
Description	Description	Multi Line	No	—	Short description of the attachment
Claim Request	Claim_Request	Lookup (Claim Request)	No	Select existing claims	Links the attachment to claim requests

4.4 Process and Workflow Design

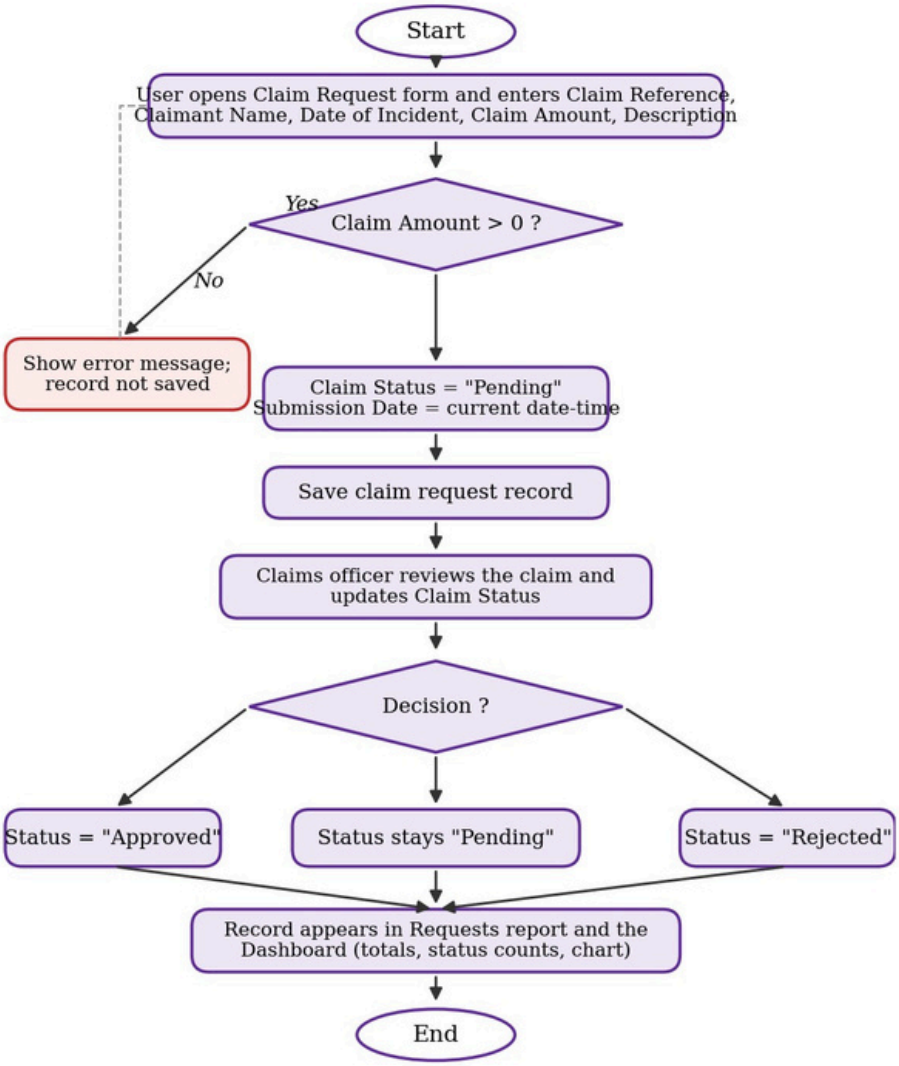


Figure 4.3: Workflow / Process Flow for Claim Request Entry and Status Tracking

Figure 4.3 shows the process followed when a claim request is entered. The validation step runs when the Claim Request form is submitted and stops invalid entries before the record is saved. The saved claim receives its status and submission date, and the record becomes visible in the Requests report and the dashboard. The claims officer later reviews the claim and updates its status. The overall process is: add customer profile → submit claim request → attach supporting documents → review the claim → update Claim Status → view reports and dashboard summaries.

4.5 User Interface and Navigation Design

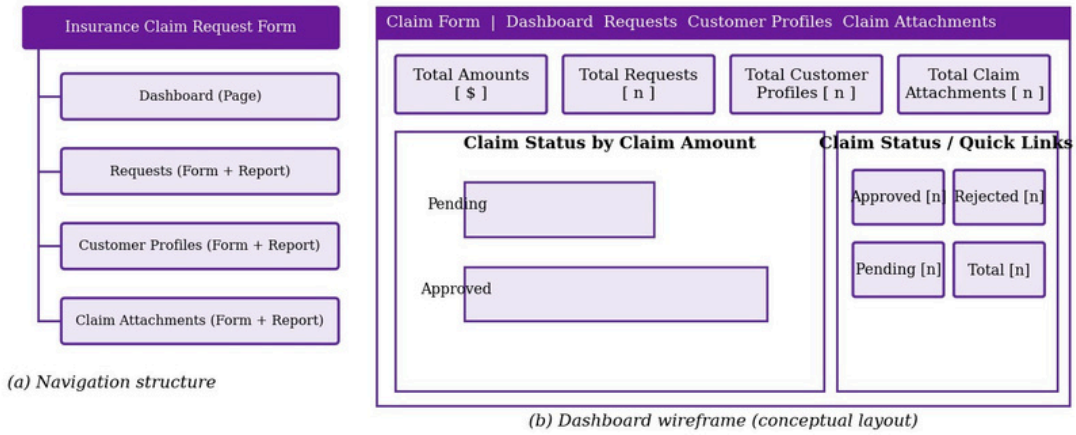


Figure 4.4: Navigation Structure and Wireframe of the Dashboard Page

The application is organised into four navigation sections that match its modules (Figure 4.4a): Dashboard, Requests, Customer Profiles and Claim Attachments. The Dashboard is the first section so that the claims officer sees the summary on opening the application. The wireframe in Figure 4.4b is the conceptual layout of the dashboard: four summary cards above a Claim Status by Claim Amount chart and a panel with the claim status counts and quick links, using a clean orange-and-white theme. It is a design sketch; the implemented page is shown in Figure 5.1.

4.6 Access Control Design

Table 4.5 defines the intended access design using A = Add, E = Edit, V = View, D = Delete and — = No access. The administrator (application owner) has full access; a claims officer role, if configured, would add, edit and view records, with deletion reserved for the administrator to prevent accidental loss of data. Customers and portal users have no access because the application does not provide a customer portal. In the current version the claims officer uses the application through the owner account, and the custom role shown in the matrix has not been configured (see Section 5.8).

Table 4.5: Role–Permission Matrix

Component	*Administrator (Application Owner)*	*Claims Officer*	*Customer*	*Portal User*
Form: Claim Request	A E V D	A E V	—	—

Component	*Administrator (Application Owner)*	*Claims Officer*	*Customer*	*Portal User*
Form: Customer Profile	A E V D	A E V	—	—
Form: Claim Attachment	A E V D	A E V	—	—
Report: Requests	V	V	—	—
Page: Dashboard	V	V	—	—

CHAPTER 5

IMPLEMENTATION

5.1 Application Setup

The Insurance Claim Request Form was created as a new application in the Zoho Creator account of the developer using the visual Form Builder and Page Builder. The application is organised into four navigation sections that correspond to its modules: Dashboard, Requests, Customer Profiles and Claim Attachments (Figure 4.4). A clean orange-and-white theme with readable labels was used to keep the interface simple for a claims officer (Figure 5.1).

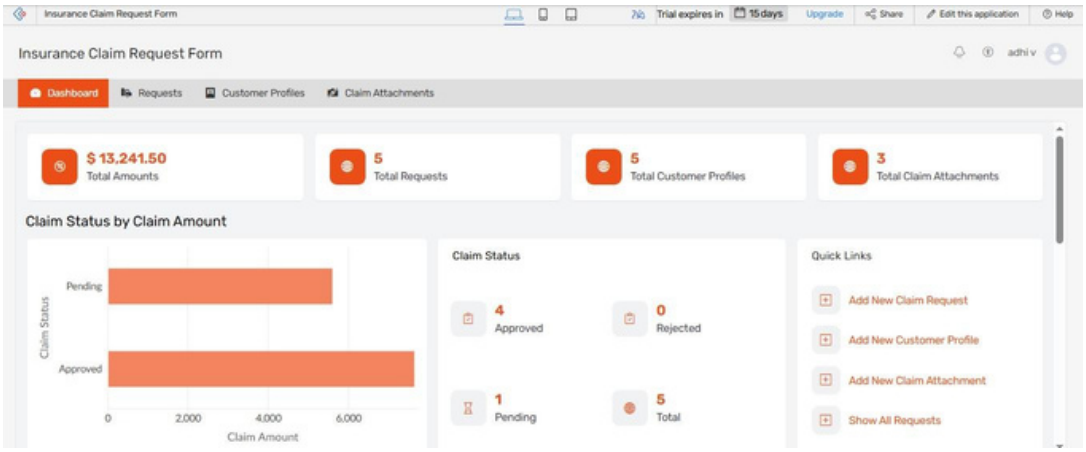


Figure 5.1: Application Home (Dashboard) and Navigation Sections

5.2 Forms and Data Capture

The application uses three forms, designed as specified in Tables 4.2 to 4.4. Each submitted form creates a record in the application database, which then appears in the corresponding report.

Claim Request form. Captures the Claim Reference, Claimant Name, Date Of Incident, Claim Amount, Incident Description, Claim Status, Supporting Document, Submission Date, Policy Number Ref and the Customer Profile lookup. The Date Of Incident field provides a date picker in the dd-MMM-yyyy format and the Claim Amount field is a currency field shown in dollars. The Claim Status is chosen from a drop-down list, and the Customer Profile lookup links the claim to existing customer profiles (Figure 5.3).

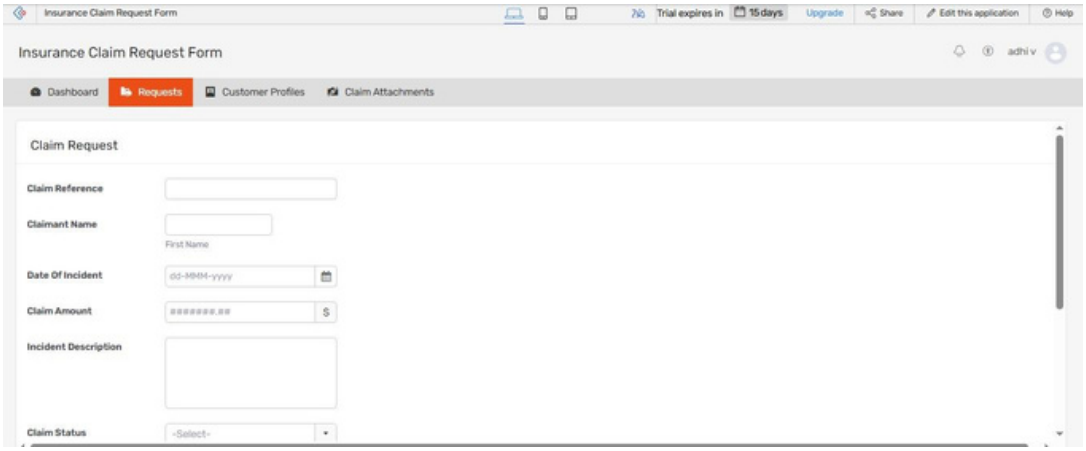


Figure 5.3: Claim Request Form in Live Mode

Customer Profile form. Stores the customer profiles identified bypolicy numbers such as POL-2026-001. These profiles are referenced by the Claim Requestform through a lookup field. Add one line about the other fields of your Customer Profileform.

Claim Attachment form. Stores the Attachment Name, File URL,Upload Date, File Size and Description of a document, together with a Claim Requestlookup that links the attachment to the claims it belongs to (Figure 5.5).

5.3 Reports

Zoho Creator creates a list report for each form, through which records can be viewed, searched, edited and deleted [2]. The application uses these reports for its three modules (Table 5.1). The Requests report lists each claim with its Claim Reference, Claimant Name, Date Of Incident, Claim Amount and Incident Description, and opening a record shows the full Overview including the Claim Status, Submission Date, Policy Number Ref and linked

Customer Profile (Figure 5.4). The Claim Attachments report lists the attachment records with the linked Claim Request (Figure 5.5).

Table 5.1: Reports Implemented

Report Name	*Type*	*Source Form*	*Filter / Grouping / Sorting*	*Custom Actions*	*Roles*
All Requests	List	Claim Request	Sorted by Claim Reference (newest first)	None	Claims Officer
All Customer Profiles	List	Customer Profile	—	None	Claims Officer
All Claim Attachments	List	Claim Attachment	—	None	Claims Officer
Claim Status by Claim Amount	Chart (Bar)	Claim Request	Grouped by Claim Status; sum of Claim Amount	None	Claims Officer (on Dashboard)

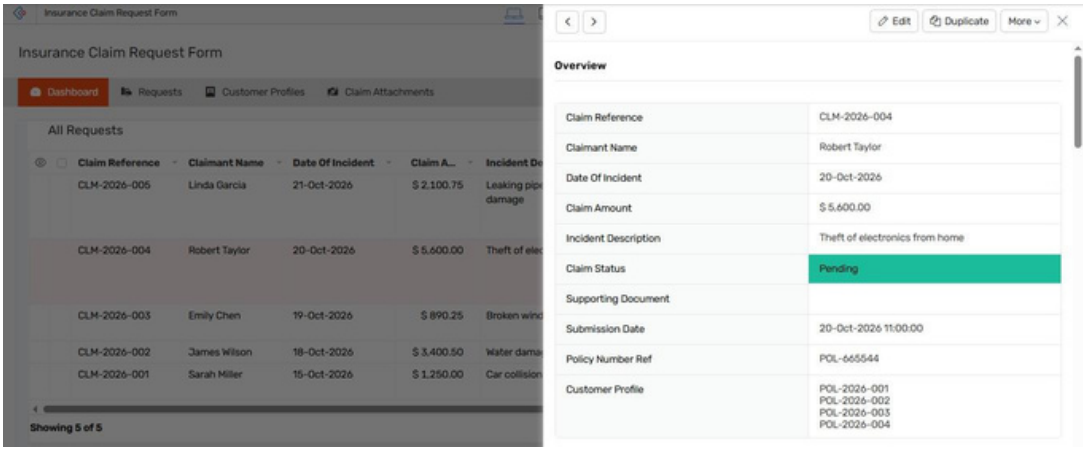


Figure 5.4: Requests Report with the Overview of Claim CLM-2026-004

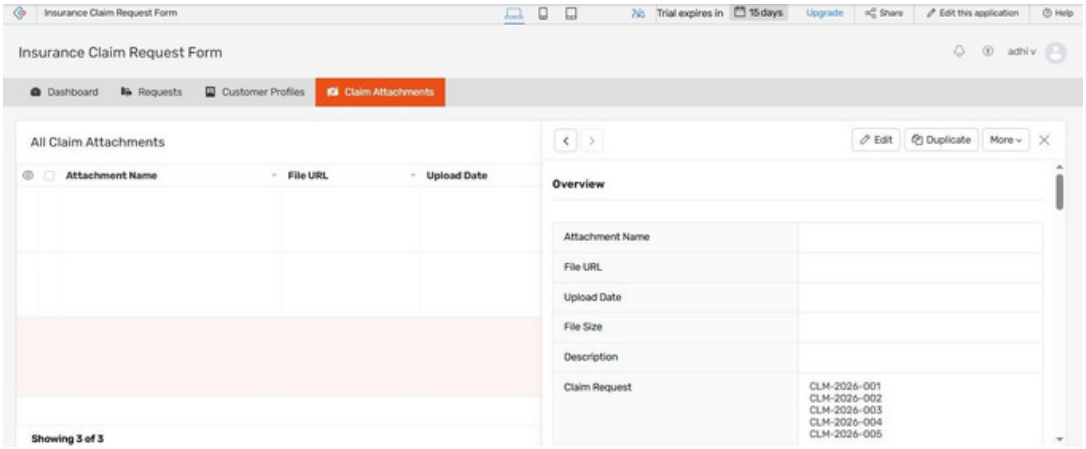


Figure 5.5: Claim Attachments Report with Record Overview

5.4 Pages and Dashboards

The Dashboard page gives the claims officer an overview of the application's records (Figure 5.1). It contains four summary cards — Total Amounts ($ 13,241.50 for the sample data), Total Requests (5), Total Customer Profiles (5) and Total Claim Attachments (3) — followed by a Claim Status by Claim Amount chart, a Claim Status panel and a Quick Links panel. The Claim Status panel shows the number of Approved (4), Rejected (0), Pending (1) and Total (5) claims. The Quick Links panel provides one-click access to Add New Claim Request, Add New Customer Profile, Add New Claim Attachment and Show All Requests. The page is created with the Zoho Creator page builder and is meant for the claims officer and the administrator.

5.5 Deluge Scripting

Deluge is the scripting language used in Zoho Creator to add custom logic to forms, workflows and functions [5]. The logic required by the application is small: validating the claim amount and setting the default status and submission date of a new claim. Table 5.2 lists the two scripts designed for this purpose, and Code 5.1 and Code 5.2 show their logic. Replace these scripts and Table 5.2 with the actual Deluge code and workflows from your application, then delete this note.

Table 5.2: Deluge Scripts Implemented

Script ID	*Location / Trigger*	*Purpose*	*Deluge Concepts Used*
DS-01	Claim Request → On Validate	Reject entries where Claim Amount is zero, negative or missing	if condition, null check, alert, cancel submit
DS-02	Claim Request → On Validate, after DS-01	Set Claim Status to "Pending" and Submission Date to the current date-time for a new claim	if condition, input field assignment, zoho.currenttime

Code 5.1: DS-01 – Claim Amount Validation Logic for the Claim Request Form

```
// DS-01 | Form:Claim Request | Event: On Validate
// Purpose: Prevent zero, negative or missing claim amounts
if(input.Claim_Amount == null || input.Claim_Amount <= 0)
{
 alert "Claim Amount must be greater than zero.";
 cancel submit;
}
```

Code 5.2: DS-02 – Default Status and Submission Date

```
// DS-02 | Form: Claim Request | Event: On Validate, after DS-01
// Purpose: Set default Claim Status and Submission Date
```

```
if(input.Claim_Status == null)
{
 input.Claim_Status = "Pending";
}
if(input.Submission_Date == null)
{
 input.Submission_Date = zoho.currenttime;
}
```

DS-01 runs first and stops the submission with a message whenever the claim amount is invalid, so only meaningful amounts reach the dashboard totals. DS-02 then sets the default values of a new claim. For example, a claim submitted for $ 5,600.00 with no status selected is stored as “Pending” and appears under Pending in the Claim Status panel, while a claim entered with an amount of 0 is rejected with the alert message.

5.6 Workflows and Automation

The application uses no scheduled, approval, Blueprint or batch workflows. The only automation required is the validation and default-value logicperformed when a claim request is submitted. Table 5.3 lists the workflow types andtheir status in the current version. Update this table to match the workflows configuredinyour application.

Table 5.3: Workflows Implemented

Workflow Name	***Type***	***Trigger / Event***	***Condition***	***Action(s)***	***Script Ref.***	***Status***
Claim Amount Validation Default Status & Submission Date	Form Workflow	Claim Request – On Validate	Every submission	Block invalidamounts with a message	DS-01	Enabled
	Form Workflow	Claim Request – On Validate	New record	Set Claim Statusand Submission Date	DS-02	Enabled
Approval Process	Not implemented	—	—	Not required for the current scope	—	—
Blueprint	Not implemented	—	—	Not required for the current scope	—	—
Scheduled Workflow	Not implemented	—	—	Proposed for future reminders (Section 9.2)	—	—
Batch / Report Workflow	Not implemented	—	—	—	—	—

5.7 Integrations

No integration has been implemented in the current version of the application. It does not connect to any other Zoho service or external system, does not call any REST API and does not expose a webhook. All data are entered and stored within the Zoho Creator application itself. This keeps the project focused on forms, reports and the dashboard. Possible future integrations, such as e-mail notifications on claim status changes, are listed in Section 9.2.

Table 5.4: Integrations Implemented

Integration	*Service / Endpoint*	*Method*	*Authentication*	*Data Exchanged / Purpose*
Zoho service integration	Not used	Not implemented	—	Not required for the current scope
External REST API / Webhook	Not used	Not implemented	—	No external data source is used

Code 5.3: Integration Script – Not Applicable to the Current Version

```
// Not applicable: the applicationdoes notcallanyZoho service
// or external REST API in the current version.
```

5.8 User Management and Role-Based Access

In the current version, the application is used through the Zoho account of the application owner, who also acts as the claims officer. No custom roles, permission profiles or record-level access criteria have been configured, and no customer portal is enabled — edit if you have configured any.

Implementing the role–permission matrix of Table 4.5 — a separate Claims Officer role without delete permission — is planned as a future enhancement. Until then, access to

claim and customer data depends on keeping the application private to the owner account and not publishing it publicly.

5.9 AI and Analytics Features

The application does not use Zia or any other AI feature of Zoho Creator, and no machine learning model has been trained or used — confirm and edit. Claim status tracking is a transparent, status-based summary of the stored records; the application does not decide whether a claim should be approved or rejected.

Analytics in the application are limited to descriptive summaries: the record totals, the Claim Status counts and the Claim Status by Claim Amount chart on the Dashboard (Figure 5.8). For the sample data, the chart shows that the Pending claims amount to about $ 5,600 and the Approved claims to about $ 7,640. These summaries help the claims officer see the overall picture but depend entirely on the accuracy and completeness of the entered records.

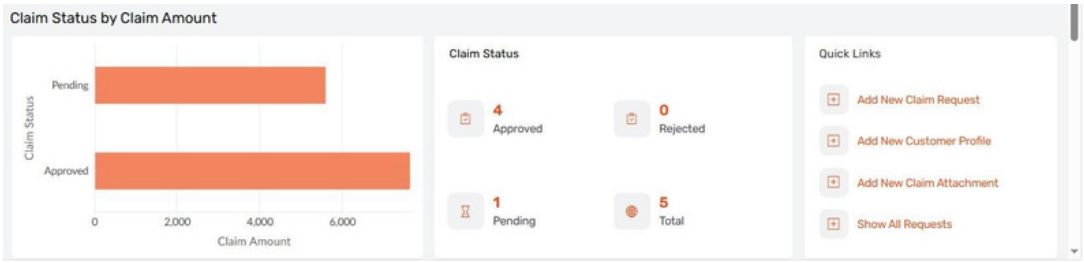


Figure 5.8: Claim Status by Claim Amount Chart on the Dashboard

5.10 Pro-Code Extension

No pro-code extension (Zoho Catalyst, custom JavaScript widgets or the Zoho Creator REST API) has been used in this application. The requirements of the project — forms,

lookups, reports, status tracking and a dashboard — are fully covered by the standard low-code features of the platform, so adding pro-code components would increase complexity without adding value at this stage.

Pro-code options could become useful in future, for example a custom widget for a more detailed claim analysis chart, the REST API for exporting claim data to another system, or a separately hosted service if a fraud-detection model is developed and evaluated later. ***5.11 Mobile and Portal Access***

The application has been designed and used through a web browser. Access through the Zoho Creator mobile app has not been tested, and no customer portal is configured, because the only intended user is the claims officer.

CHAPTER 6

TESTING AND DEBUGGING

6.1 Testing Strategy

The testing strategy for the application covers the following levels:

- ***Unit testing of forms:*** each form is tested with valid input and with missing or invalid values.
- ***Data and relationship testing:*** saved records are checked in the reports, and lookup links between customer profiles, claim requests and claim attachments are verified.
- ***Negative testing:*** zero or negative claim amounts and empty required fields are entered to confirm that they are handled as designed.
- ***Report and dashboard testing:*** saved records are checked in the reports, and the dashboard totals and status counts are compared with the records in each form.
- ***User acceptance testing:*** at least two users are to try the application and give feedback (Section 6.5).

6.2 Test Cases

Table 6.1 lists the test cases. The results marked Pass were verified from the application screens (Figures 5.1, 5.4 and 5.5); the cells highlighted in yellow are to be filled after executing the test on the live application.

Table 6.1: Test Cases

TC ID	*FR Ref.*	*Test Description*	*Input / Steps*	*Expected Result*	*Actual Result*	*Status*
TC-01	FR-01	Submit a claim request with valid information	CLM-2026-004, Robert Taylor, 20-Oct-2026, $ 5,600.00, Theft of electronics from home	Record saved and listed	Claim CLM-2026-004 saved with all entered details	Pass
TC-02	FR-01, FR -0 8	Verify the claim appears in the Requests report	Open All Requests	All claims are listed with their details	Five claims (CLM-2026-001 to 005) listed – “Showing 5 of 5”	Pass
TC-03	FR -0 1	Store a claim amount with decimals	Claim Amount: 2,100.75 (CLM-2026-005)	Amount stored and displayed as $ 2,100.75	Amount displayed as $ 2,100.75 in the report	Pass

TC ID	*FR Ref.*	*Test Description*	*Input / Steps*	*Expected Result*	*Actual Result*	*Status*
TC-04	FR -0 2	Record the claim status	Open CLM-2026-004 overview	Claim Status shown	Claim Status shown as “Pending”	Pass
TC-05	FR -0 4	Customer profiles are stored and counted	Open Dashboard	Total Customer Profiles equals the stored profiles	Dashboard shows 5 Total Customer Profiles	Pass
TC-06	FR -0 5	Link a claim to customer profiles	Open CLM-2026-004 overview	Linked profiles shown	Customer Profile shows POL-2026-001 to POL-2026-004	Pass
TC-07	FR -0 6	Claim attachments are stored and linked	Open Claim Attachments report	Attachment records listed with linked claims	3 attachment records listed; Claim Request shows CLM-2026-001 to 005	Pass
TC-08	FR-09	Total claim amount equals the sum of claims	1,250.00 + 3,400.50 + 890.25 + 5,600.00 + 2,100.75	$ 13,241.50	Dashboard Total Amounts shows $ 13,241.50	Pass
TC-09	FR-09	Dashboard counts match the records	Compare cards with report counts	Counts match the records	Total Requests 5, Customer Profiles 5, Claim Attachments 3 match the reports	Pass
TC-10	FR -1 0	Claim status counts add up to the total	Open Claim Status panel	Approved + Rejected + Pending = Total	4 + 0 + 1 = 5	Pass
TC-11	FR -1 0	Chart reflects claim amounts by status	Open Claim Status by Claim Amount chart	Pending ≈ 5,600; Approved ≈ 7,641	Pending bar ≈ 5,600 and Approved bar ≈ 7,600	Pass
TC-12	FR -0 7	Reject a zero claim amount	Claim Amount = 0	Error message; record not saved		
TC-13	FR -0 7	Reject a negative claim amount	Claim Amount = -500	Mandatory field error; record not saved		
TC-14	FR -0 1	Submit a claim with required fields left empty	Claim Reference left empty			

Elevenof the fourteen test cases (TC-01 to TC-11) were verified from the live application data;TC-12 to TC-14, which exercise the validation logic, are to be executed and recorded.

6.3Debugging Deluge Scripts

Thefollowing techniques can be used to debug the Deluge logic of the application: (i) “info”statements to print intermediate values; (ii) the alert messages of DS-01, which show the reason why a submission was rejected; (iii) the error messages displayed by Zoho Creator on form submission; and (iv) repeated testing with boundary values such as 0, a negative amount and a very large amount.

During testing, the Claim Attachments report was observed to show empty Attachment Name, File URL and Upload Date columns for the sample records (Figure 5.5), although

the records exist and are linked to claim requests. This was recorded as BUG-01 in Table 6.2. Add any other defects found while testing and describe how you fixed them.

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	***Description***	***Root Cause***	***Fix Applied***	***Status***
BUG-01	Claim Attachments report shows blank Attachment Name, File URL, Upload Date and File Size for the sample records	Sample attachment records were created with only the Claim Request lookup filled	Complete the attachment fields for the sample records	Open / Closed
BUG-02				

6.5UserAcceptance Testing

Peerreviewers are asked to testthe application in the claims officer role by adding sample customer profile, claim requestand attachment records and reviewing the reports and the Dashboard. Their feedback andthe resulting actions are recorded in Table 6.3. Fill this table after the reviewers have tested the application.

Table 6.3: User Acceptance Feedback

User	***Role Tested***	***Feedback***	***Action Taken***
Reviewer 1 (name, class)	Claims Officer		
Reviewer 2 (name, class)	Claims Officer		

CHAPTER 7

DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Zoho Creator supports separate Development, Stage and Production environments in its supported plans, allowing changes to be tested before they are published to users. The availability of these environments depends on the Zoho Creator edition being used.

For this application, development, testing and verification were carried out in the primary Zoho Creator Development environment under the developer's account in October 2026, on the Zoho Creator trial edition. Because the application is intended for a single claims officer (the application owner), it is accessed directly within the authenticated Zoho Creator workspace without requiring separate multi-stage enterprise environment promotion.

Table 7.1: Deployment Checklist

S.No.	***Checklist Item***	***Done (✓)***
1	All test cases executed in Development / Stage	✓ (TC-01 to TC-11); TC-12 to TC-14 pending
2	Sample/test data removed or isolated	✓ (Fictional demonstration data)
3	Roles, permissions and users configured in Production	N/A (Single-owner access)
4	Connections authorised in Production	N/A (no connections)
5	Schedules and workflows enabled	✓ (Claim Request On Validate workflow enabled) ✓ (Verified in owner workspace)
6	Application published and URL verified	

S.No.	Checklist Item	Done (✓)
7	Version / release notes recorded	✓ (Version 1.0 – October 2026)

7.2Performance Optimisation

Theapplication stores a small data set, so performance was not a major concern and no measurements were taken. The following good practices were nevertheless followed in the design: each form contains only the fields that are needed; the designed scripts operate only on the submitted record and do not fetch other records inside loops; and the reports show only the columns needed for review. Table 7.2 lists these practices; before-and-after values are not reported because they were not measured.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	Af ter	Improvement
Minimal fields per form	All three forms	Not measured	Not measured	Simpler data entry
No fetch statements inside loops	DS-01, DS-02 (design reference)	Not measured	Not measured	Logic limited to the submitted record
Limited columns in the reports	Requests and Claim Attachments reports	Not measured	Not measured	Easier review of records

7.3 Security Features

The security measures relevant to the application are basic and are listed in Table 7.3. Access to the application requires signing in with the appropriate Zoho account, claim and customer information should not be exposed unnecessarily, and validation of the claim amount protects the integrity of the data. Record-level permissions and custom roles are not configured in the current version.

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Zoho account sign-in	Access to the whole application	Unauthorised access
Application kept private to the owner account (not published publicly)	Application sharing settings	Unnecessary exposure of claim and customer information
Validation of the claim amount	Claim Request form	Invalid or inconsistent data
Record-level / role-based permissions	Not configured (future enhancement)	Accidental editing or deletion of records

7.4 Governance and Administration

Governance of the application is kept simple because it is a single-owner academic application. Consistent naming is used for forms and fields (for example, Claim_Reference, Claim_Amount, Claim_Status), which makes the data model and any scripts easier to read. The application owner is responsible for maintaining the forms, correcting data-entry mistakes and keeping the sample data separate from any real data. Advanced enterprise administration features such as automated audit trails and scheduled cloud backups were not required for this small-scale academic project; standard Zoho Creator application settings and manual CSV/spreadsheet export from list reports are used when record backups are needed.

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Application Output

The working of the application is demonstrated using one scenario with fictional sample data. The claims officer opens the Requests section and the Claim Request form (Figure 8.1) and records a claim for Robert Taylor with the reference CLM-2026-004: Date Of Incident 20-Oct-2026, Claim Amount $ 5,600.00, Incident Description “Theft of electronics from home”, Policy Number Ref POL-665544 and Claim Status Pending (Figure 8.2).

The officer then opens the Dashboard (Figure 8.3) to confirm that the totals have been updated: with the five sample claims the application shows a Total Amount of $ 13,241.50, 5 Total Requests, 5 Total Customer Profiles and 3 Total Claim Attachments. The Claim Status panel shows 4 Approved, 0 Rejected, 1 Pending and 5 Total claims, and the chart shows the Pending and Approved claim amounts.

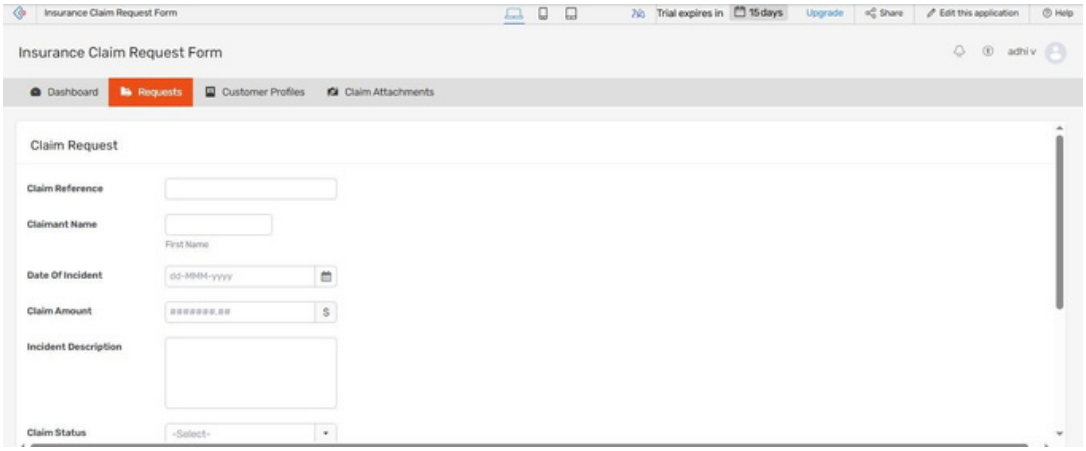


Figure 8.1: Scenario Step 1 – Opening the Claim Request Form

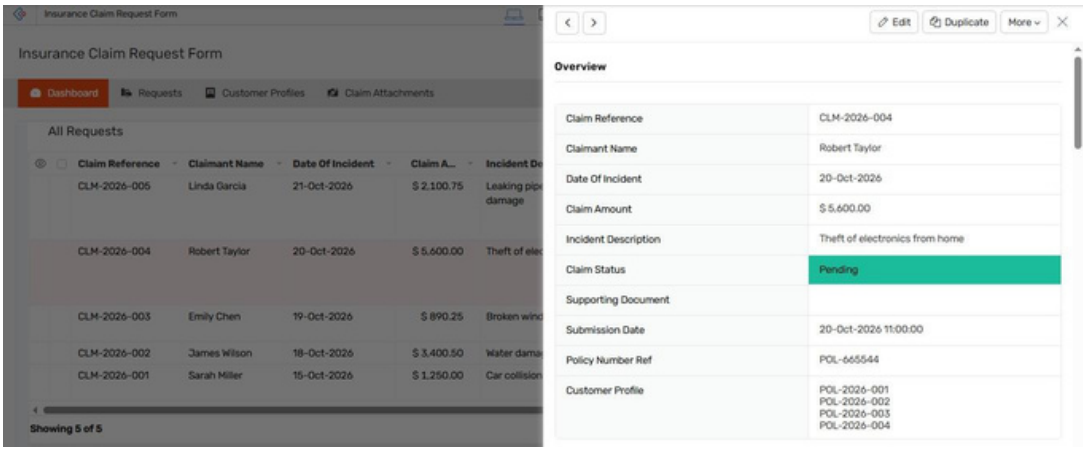


Figure 8.2: Scenario Step 2 – Claim CLM-2026-004 Recorded with Status Pending

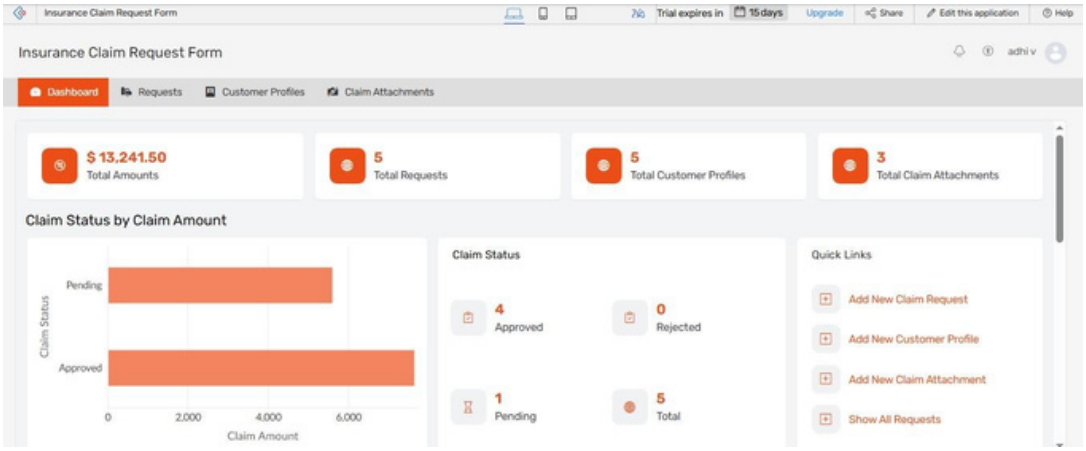


Figure 8.3: Scenario Step 3 – Dashboard Totals, Claim Status and Chart after the Update

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	*Implemented Through*	*Outcome / Evidence*
To create a centralised form for recording claim requests	Section 5.2 – Claim Request form	Claim Request form and Requests report implemented (Figures 5.3, 5.4)
To maintain customer profiles and link them to claims	Sections 5.2 and 4.2 – Customer Profile form and lookup	Five profiles stored; lookup shown in the claim overview (TC-05, TC-06)
To store claim attachment details linked to claims	Section 5.2 – Claim Attachment form	Three attachment records linked to claim requests (Figure 5.5, TC-07)
To record and track the status of each claim	Section 5.2 – Claim Status field	Claim Status of each claim stored and summarised (TC-04, TC-10)
To provide reports and a dashboard with totals and status counts	Sections 5.3 and 5.4	Dashboard totals, status counts and chart configured (Figures 5.1, 5.8; TC-08 to TC-11)
To demonstrate Zoho Creator low-code features	Chapters 4 and 5	Application built mainly through visual configuration (Section 8.5)

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	*Existing System*	*Proposed Application*	*Improvement*
Recording one claim	Details written or typedin a register or spreadsheet	Details entered once in a form with a date picker, currency field and drop-down status	Consistent data entry
Information storage	Separate files for claims, policies and documents	Related forms linked by lookups in one application Dashboard cards and Claim	Information kept in one place
Totals and status counts	Counted by hand	Status panel generated from stored records	Fewer manual steps (time not measured)
Data accessibility	Records available only wherethe file or register is kept	Available through a browser with the authorised Zoho account	Easier access for the authorised user

8.4 Limitationsof theLow-Code Approach

The following limitations of the current application and of the low-code approach were identified:

- ***Simple status tracking:*** the Claim Status is chosen by the claims officer; the application does not evaluate a claim or decide on its approval.
- ***Data dependence:*** the dashboard depends entirely on the accuracy and completeness of the entered records.
- ***No AI or machine learning:*** the current system does not use a trained model and does not detect fraud or predict claim outcomes.
- ***Platform dependence:*** availability, deployment options and some features depend on the configured Zoho Creator account and edition (the application was built on a trial edition), and the application cannot easily be moved to another platform.
- ***Customisation limits:*** the layout and charts are limited to the components offered by the Zoho Creator builder; highly customised visualisations would require pro- code extensions.
- ***Access control:*** custom roles and record-level permissions are not yet configured.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

No-code. Most of the application sits at the no-code end of the spectrum: the three forms, their field types and lookups, the list reports and the dashboard are all configured visually without writing code.

Low-code. The validation of the claim amount and the default status and submission date require a small amount of logic, expressed as short Deluge scripts (Section 5.5). This is the low-code part of the application.

Pro-code. No part of the application currently needs pro-code. Moving to pro-code would be justified only if the application required features beyond the platform, such as a custom interactive visualisation, integration with an insurer's core system or a fraud-detection model developed and evaluated separately.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Insurance Claim Request Form demonstrates how a low-code platform can support basic claim request management and status tracking. Using Zoho Creator, the project combines structured data entry for claim requests, customer profiles and claim attachments with lookup relationships, reports and a dashboard that summarises the total claim amount, record counts and claim status in one application.

The project has strong educational value: it involved data modelling, form design, report and dashboard creation, test execution and reflection on the strengths and limits of low-code development. The dashboard is intentionally simple and transparent; it helps a claims officer see the state of all claims at a glance while the decision on each claim remains with the officer.

9.2 Future Enhancements

- E-mail or in-app notifications to the claimant and the claims officer when the Claim Status changes (proposed).
- An approval process so that high-value claims are reviewed by a supervisor before approval (proposed).
- Secure role-based access for different users, implementing the matrix of Table 4.5 (proposed).
- A customer portal through which policyholders can submit claims and track their status (proposed).
- Mandatory-field and date validations, such as preventing an incident date later than the submission date (proposed).
- Direct file upload for claim attachments with the file name and size captured automatically (proposed).
- Additional reports and charts, such as claims by month or by incident type (proposed).

- Integration with other Zoho services or an accounting system for claim payments, and a fraud-detection model only if it is developed and properly evaluated (proposed).

9.3 Learning Outcomes and Career Relevance

Building the Insurance Claim Request Form strengthened skills in designing a data model, configuring forms with appropriate field types and lookups, creating reports and a dashboard, writing simple validation logic, and planning test cases with positive and negative values. It also showed that low-code development still requires careful thinking about data quality, validation and access control.

These skills relate directly to roles such as citizen developer, low-code application developer, solutions consultant and business process automation engineer, where business requirements must be converted quickly into working applications. Knowing when a low-code solution is sufficient — and when pro-code is needed — is also valuable for a future career in software development and in artificial intelligence and machine learning, where data collection and organisation are the first steps of any analysis.

REFERENCES

[1] A. C. Bock and U. Frank, “Low-code platform,” Business & Information Systems Engineering, vol. 63, no. 6, pp. 733–740, Dec. 2021, doi: 10.1007/s12599-021-00726-8.

[2] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/

[3] A. Sahay, A. Indamutsa, D. Di Ruscio, and A. Pierantonio, “Supporting the understanding and comparison of low-code development platforms,” in Proc. 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA), Portorož, Slovenia, 2020, pp. 171–178, doi: 10.1109/SEAA51224.2020.00036.

[4] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, “Low-code development and model-driven engineering: Two sides of the same coin?,” Software and Systems Modeling, pp. 437–446, 2022, doi: 10.1007/s10270-021-00970-2.

[5] Zoho Corporation, “Deluge Help.” [Online]. Available: https://www.zoho.com/deluge/help/

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Cells highlighted in yellow depend on how your application is configured; confirm them before submission and update the total.

Table A.1: Feature Coverage Checklist

Unit	*Zoho Creator Feature*	*Done*	*Where in the Application*	*Report Section*	*Figure / Code No.*
I	Application created & sections/menu organised	Y	Four navigation sections	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	Y	Single Line, Name, Date, Currency, Multi Line, Drop Down, Date-Time, File Upload, Lookup (confirm against your forms)	5.2	Tables 4.2–4.4
II	Lookup field (relationship between forms)	Y	Claim Request → Customer Profile; Claim Attachment → Claim Request	4.2, 5.2	Table 4.1
II	Subform (parent–child data)	N	Not used	4.2, 5.2	—
II	Field validations / layout rules	Y	Claim Request – claim amount validation	5.2	Code 5.1
II	List report with filter, grouping, custom action	N	List reports used; custom actions not used	5.3	Table 5.1
II	Kanban / Calendar / Timeline / Map report	N	Not used	5.3	—
II	Pivot table / chart report	Y	Claim Status by Claim Amount chart on the Dashboard	5.3	Fig. 5.8
II	Page / dashboard built with UI Builder	Y	Dashboard page	5.4	Fig. 5.1
II	Deluge – form events (validate / success / field actions)	Y	Claim Request – On Validate (DS-01, DS-02)	5.5	Code 5.1, 5.2
II	Deluge – functions, fetch, loops, collections	N	No functions, fetch or loops required	5.5	—
II	Email / in-app notifications	N	Not implemented (future enhancement)	5.5, 5.6	—
II	Form workflow	Y	Claim Request form workflow	5.6	Table 5.3
II	Scheduled workflow	N	Not implemented	5.6	—
II	Approval process	N	Not implemented	5.6	—
II	Blueprint	N	Not implemented	5.6	—
II	Batch / report workflow	N	Not implemented	5.6	—
II	Testing & debugging evidence	Y	Test cases and defect log recorded	6	Table 6.1
III	Integration with a Zoho service	N	Not implemented	5.7	—

Unit	*Zoho Creator Feature*	*Done*	*Where in the Application*	*Report Section*	*Figure / Code No.*
III	Connection / REST API call (invokeurl)	N	Not implemented	5.7	—
III	Webhook or Creator REST API	N	Not implemented	5.7	—
III	Users, roles and permission profiles	N	Not configured (future enhancement)	5.8	Table 4.5
III	Record-level access criteria / customer portal	N	Not configured; no portal	5.8	—
III	Zia / AI feature	N	Not used	5.9	—
III	Analytics (charts / Zoho Analytics)	Y	Dashboard totals, status counts and chart (descriptive only)	5.9	Fig. 5.8
IV	Performance optimisation applied & measured	N	Good practices followed; not measured	7.2	Table 7.2
IV	Deployment across environments	Y	Development environment in Zoho Creator	7.1	Fig. 7.1
IV	Security controls (encryption, OAuth, permissions)	N	Basic controls only (Zoho sign-in, validation)	7.3	Table 7.3
IV	Governance (audit trail, administration)	Y	Application naming conventions and settings	7.4	—
IV	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used (justified in Section 5.10)	5.10	—
V	Low-code limitations & spectrum analysis	Y	Limitations and spectrum discussed	8.4, 8.5	Sec. 8.4, 8.5
V	Mobile app / portal access	N	Not tested / not configured	5.11	—
—					

Total features implemented:__ / 32 (update after confirmingthe highlighted rows).

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	***Course Outcome Statement***	***Evidenced In***	***Self-Assessed Level***	***Examiner Level***
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2		
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6		
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9		
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7		
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3		

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Claim Request Form – On Validate (DS-01 andDS-02)

Replace with the actual Deluge code from your application.

Code C.1: Claim Request – On Validate

```
// Script ID : DS-01, DS-02
// Location : Claim Request form -> On Validate
// Purpose   : Reject invalid claim amounts; set default Claim Status
//             and Submission Date for a new claim.

// --- DS-01: Claim Amount Validation Logic ---
if(input.Claim_Amount == null || input.Claim_Amount <= 0)
{
 alert "Claim Amount must be greater than zero.";
 cancel submit;
}

// --- DS-02: Default Status and Submission Date ---
if(input.Claim_Status == null)
{
 input.Claim_Status = "Pending";
}
if(input.Submission_Date == null)
{
 input.Submission_Date = zoho.currenttime;
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open a web browser, sign in to Zoho Creator with the authorised Zoho account and open the Insurance Claim Request Form application from the list of applications or via the application URL given in Project at a Glance. The Dashboard is shown first; the other sections are available in the navigation menu. Mobile access has not been tested in the current version.

D.2 Instructions for the Claims Officer

- Open Customer Profiles and add each customer profile with its policy number; use the report to view, edit or search records.
- Open Requests and select the Claim Request form (or use the Add New Claim Request quick link on the Dashboard).
- Enter the Claim Reference, Claimant Name, Date Of Incident, Claim Amount and Incident Description; select the Claim Status, Policy Number Ref and Customer Profile; attach a Supporting Document if available, and submit.
- If an error message appears, correct the entries and submit again.
- Open Claim Attachments and add the Attachment Name, File URL, Upload Date, File Size and Description, and select the Claim Request to which the attachment belongs.
- To change the outcome of a claim, open the record in the Requests report, choose Edit and update the Claim Status to Approved or Rejected.
- Open the Dashboard to view the total amount, the number of requests, customer profiles and attachments, the Claim Status counts and the Claim Status by Claim Amount chart.

D.3 Instructions for the Application Administrator

- Sign in with the application owner account and open the application in edit mode to modify forms, fields, reports or the Dashboard.
- After any change, repeat the relevant test cases in Table 6.1 before using the application with real data.

- Keep the application private to authorised users and avoid publishing it publicly, because it contains customer and claim information.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	[Student Name]	Register No.	[Register No.]
Project Title	INSURANCE CLAIM REQUEST FORM	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	***Criterion***	***Indicators***	***Max. Marks***	***Marks Awarded***
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model,form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts, form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		100	

Table E.2: Performance Level Descriptors

Level	***% of Max. Marks***	***Descriptor***
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

SignatureofInternalExaminer

Name:

Signature of External Examiner

Name:

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	[Student Name]	Register No.	[Register No.]
Department / Section	Artificial Intelligence and Machine Learning	Date of Viva	
Application URL Verified	Yes / No	Demo Duration	__ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	***Question / Topic Asked***	***Unit***	***Response (E / G / S / NI)***	***Remarks***
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	***Max. Marks***	***Marks Awarded***
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	100	

Examiners' Remarks:

INTERNAL EXAMINER ***EXTERNAL EXAMINER***

Name: Name: